Laborator 4 - MAP

Metode Avansate de Programare, secția Informatică 2026–2027

DEADLINE: Săptămâna 6

Precondiții

- Laboratorul 4 continuă proiectul **Rețea de socializare** (vezi fișierul **Enunt Proiect Lab - Retea de socializare.pdf**) și pornește de la soluția laboratorului 3 (persistență în fișier text, `RepositoryFactory`, `config.properties`).
- Materiale: Cursul 4-5 (Java 8: metode default și statice în interfețe, interfețe funcționale, expresii lambda, referințe la metode, `Optional`, Stream API) și Seminarul 5 (`Validator` ca lambda și `andThen`, `findAll(Predicate)`, `CriteriiUtilizator`, comparatori, `RepositoryFactory` cu `Supplier`).
- Laboratorul 4 este un laborator de **refactorizare**: aplicația face exact ce făcea la laboratorul 3, dar codul se rescrie în stil Java 8. Buclele clasice dispar, comportamentul se transmite ca parametru (lambda, referințe la metode), iar prelucrările de colecții se scriu cu stream-uri.

A. Cerințe funcționale

- Nu sunt cerințe funcționale noi pentru acest laborator.
- Funcționalitățile din laboratorul 2 (add/remove utilizator, add/remove prieten, numărul de comunități, cea mai sociabilă comunitate) trebuie să funcționeze în continuare, cu datele păstrate în fișier și cu aceleași rezultate ca înainte de refactorizare.

B. Cerințe non-funcționale – SUGESTII DE IMPLEMENTARE

1. Interfețe funcționale și expresii lambda (2.5p)

- `Validator<T>` este adnotată cu `@FunctionalInterface` și are metoda default `andThen(Validator<T> other)`. Cel puțin o regulă de validare nouă se scrie ca lambda și se compune cu validatorul existent, fără o clasă nouă și fără să modificați `UtilizatorValidator`. (0.75p)
- `Repository<ID, E>` are metoda default `List<E> findAll(Predicate<E> filter)`. Criteriile de căutare sunt metode statice care întorc `Predicate<Utilizator>` (de exemplu, în `CriteriiUtilizator`) și se combină cu `and`, `or`, `negate`, `Predicate.not`. Nu se adaugă metode de tipul `findByLastName` în repository. (0.75p)
- `RepositoryFactory` nu mai folosește `switch`: fiecărui `RepositoryType` îi corespunde, într-un `Map`, un `Supplier` (sau un `Function<Validator<E>, Repository<ID, E>>`) care creează repository-ul doar pentru tipul ales. Se aplică pentru ambele repository-uri (utilizatori și prietenii). (0.5p)
- Conversia linie ↔ entitate din repository-urile pe fișier: fie se păstrează Template Method (`AbstractFileRepository` cu subclase), fie se trece la un singur `FileRepository<ID, E>` care primește prin constructor `Function<String, E>` și `Function<E, String>` (compoziție în loc de moștenire). Alegerea se justifică în `readme.txt`. (0.5p)

2. Referințe la metode (1p)

- Lambda-urile care doar apelează o metodă existentă (`x -> f(x)`) se înlocuiesc cu referințe la metode. În proiect apar toate cele patru tipuri, acolo unde au sens: metodă statică (`CriteriiUtilizator::faraPrieteni`), metodă de instanță a unui obiect anume (`System.out::println`), metodă de instanță a unui obiect oarecare (`Utilizator::getLastName`) și constructor (`Utilizator::new`, `ArrayList::new`, `InMemoryRepository::new`). (1p)

3. Stream API (3.5p)

- În service, operațiile pe colecții se scriu cu stream-uri, fără `for`/`while`: prietenii unui utilizator, ștergerea unui utilizator împreună cu prieteniile lui, numărul de comunități și cea mai sociabilă comunitate. Se folosesc operațiile din curs: `filter`, `map`, `flatMap`, `sorted`, `distinct`, `count`, `anyMatch`/`allMatch`/`noneMatch`, `max`/`min`, `reduce`, `collect`. (1.5p)
- Rezultatele se construiesc cu `collect` (`Collectors.toList`, `toSet`, `toMap`, `groupingBy`, `counting`, `joining`), nu prin adăugare într-o listă exterioară din `forEach`. Funcțiile transmise lui `filter` și `map` nu au efecte secundare (nu modifică date din afara stream-ului). (0.75p)
- Sortările folosesc `Comparator.comparing`, `comparingInt`, `thenComparing`, `reversed`, fără `compare` scris de mână și fără scăderi de tipul `(a, b) -> a - b`. (0.5p)
- Repository-urile pe fișier citesc cu `Files.lines(path)` (într-un try-with-resources, pentru că stream-ul ține fișierul deschis) și scriu cu `Files.write(path, linii)`, unde `linii` se obține printr-un stream peste entități. Formatul fișierelor din laboratorul 3 nu se schimbă. (0.75p)

4. Optional (1p)

- Operațiile care pot să nu aibă rezultat întorc sau folosesc `Optional`: căutarea unui utilizator în service (`Optional.ofNullable(repo.findOne(id))`), rezultatul lui `max`, `min`, `reduce`, `findFirst`. Valoarea se extrage cu `orElseThrow`, `orElse`, `ifPresent`, `map`; nu se apelează `get()` fără verificare. (0.75p)
- Decideți dacă interfața `Repository` întoarce în continuare `null` sau trece la `Optional<E>` (ca în exemplul `AbstractRepository` din curs). Dacă o schimbați, se modifică toate implementările și Javadoc-ul, consecvent. Alegerea se justifică în `readme.txt`. (0.25p)

5. Comportament neschimbat și documentație (1p)

- Aplicația produce aceleași rezultate ca la laboratorul 3 pe aceleași fișiere de date: fișierul lipsă, liniile corupte (raportate cu numărul liniei) și excepțiile se tratează la fel. Excepțiile checked aruncate în interiorul unei lambda (de exemplu `IOException`) se împachetează în `RepositoryException` sau `UncheckedIOException`. (0.5p)
- Javadoc în engleză pentru metodele și interfețele noi (`@param`, `@return`, `@throws`), inclusiv pentru metodele default și pentru parametrii de tip `Predicate`, `Function`, `Supplier`. (0.5p)

6. Demonstrație la predare (1p)

- Se arată: aplicația rulează cu aceleași rezultate ca la laboratorul 3; cel puțin trei locuri din cod, înainte (laboratorul 3) și după refactorizare, cu explicarea pipeline-ului de stream (ce este operație intermediară, ce este operație terminală); adăugarea unei reguli de validare și a unui criteriu de căutare noi doar prin lambda, fără clase noi. (1p)

Sugestii și observații

- Porniți de la codul de la Seminarul 5. Atenție la:
  - `findAll()` întoarce `Iterable`, care nu are `stream()`: folosiți `StreamSupport.stream(repo.findAll().spliterator(), false)`, scris o singură dată, într-o metodă ajutătoare;
  - un stream se consumă o singură dată: nu îl păstrați într-un câmp și nu îl refolosiți;
  - variabilele locale folosite într-o lambda trebuie să fie effectively final; un contor modificat din `forEach` nu compilează, iar asta indică de obicei că trebuie folosit `count()`, `sum()` sau `collect`;
  - `Comparator.comparingInt(u -> u.getFriends().size()).reversed()` nu compilează direct în `sort(...)`: compilatorul nu deduce tipul lui `u` (vezi Seminarul 5); dați tipul explicit sau folosiți o variabilă intermediară;

  - pentru a număra liniile corupte cu numărul liniei, citiți liniile într-o listă și parcurgeți indicii cu `IntStream.range(0, linii.size())`;
  - `removeIf` șterge corect dintr-o colecție; `lista.remove(x)` dintr-un for-each aruncă `ConcurrentModificationException` (sau, mai rău, nu aruncă și sare un element).
- Comunitățile (componentele conexe) se pot obține cu o parcurgere DFS/BFS în care vecinii se procesează cu stream-uri (`filter(v -> !vizitat.contains(v))`), sau atribuind fiecărui utilizator id-ul componentei și grupând apoi cu `Collectors.groupingBy`. Cea mai sociabilă comunitate: `max(Comparator.comparingInt(...))` peste comunități.
- IntelliJ semnalează bucle care pot fi înlocuite cu stream-uri (inspecția „Loop can be replaced with Stream API"). Folosiți-o, dar verificați că rezultatul se citește mai ușor decât bucla.
- `Optional` se folosește ca tip de retur, nu pentru câmpuri sau parametri.
- `parallelStream()` nu aduce nimic pentru datele din acest proiect și devine periculos dacă funcțiile au efecte secundare.

Livrabile

- Codul sursă (proiect Gradle / IntelliJ), împreună cu `config.properties` și folderul `data/` (inclusiv fișierul de test cu linii corupte de la laboratorul 3).
- Un document `readme.txt` în care precizați: un tabel înainte/după pentru cel puțin cinci bucle înlocuite (clasa, metoda, ce operații de stream s-au folosit); ce interfețe funcționale ați folosit și unde; alegerea pentru `Optional` în `Repository` și pentru Template Method vs. `Function` în repository-urile pe fișier; eventualele bucle păstrate și de ce.

Se va folosi un sistem de versionare, GIT, vezi cursul 4.

Restricții de implementare

- În pachetele `domain`, `service`, `repository` și în UI nu se mai folosesc bucle clasice (`for` cu index, for-each, `while`, `do-while`) pentru parcurgerea colecțiilor. Excepție acceptată doar cu justificare în `readme.txt` (de exemplu, un algoritm de backtracking pentru cel mai lung drum, unde recursivitatea cu stream-uri devine greu de citit).
- Nu se folosesc clase anonime acolo unde o lambda sau o referință la metodă este suficientă.
- Nu se modifică formatul fișierelor de date și nu se implementează încă persistența în baza de date: este cerința laboratorului 5. Pentru `DATABASE`, repository-urile rămân schelete care aruncă `UnsupportedOperationException`.
- Nu se folosesc biblioteci externe (Guava, Vavr, StreamEx etc.): doar `java.util.function`, `java.util.stream`, `java.util.Optional` și `java.nio.file`.